



南開大學
Nankai University

计算机学院
计算机体系结构实验报告

例外中断

姓名：王一诺

学号：2312331

专业：计算机科学与技术

指导老师：董前琨

2025 年 11 月 20 日

目录

1	实验要求	2
2	系统调用例外	2
2.1	设计思路	2
2.2	核心代码与说明	2
2.3	测试程序与结果分析	4
3	地址错例外	4
3.1	设计思路	4
3.2	核心代码与说明	5
3.3	测试程序与结果分析	6
4	整形溢出例外	8
4.1	设计思路	8
4.2	核心代码与说明	8
4.3	测试程序与结果分析	9
5	断点例外	10
5.1	设计思路	10
5.2	核心代码与说明	10
5.3	测试程序与结果分析	11
6	保留指令例外	12
6.1	设计思路	12
6.2	核心代码与说明	12
6.3	测试程序与结果分析	13
7	中断例外	14
7.1	设计思路	14
7.2	核心代码与说明	14
7.3	测试程序与结果分析	16
8	总结感想	17

1 实验要求

请根据《CPU 设计实战》中的第七章内容，完成对五级流水线 CPU 的例外检测、例外实现，完成 7.4 的实验任务：

1. 有 syscall 例外支持；
2. 自行添加其他例外支持

要求：

1. 自行修改 inst_rom 中的指令，复现例外；
2. 实现对应的例外支持；
3. 实验报告中体现实现支持之前的波形，和之后的波形，并详细分析实验结果。

接下来我将对每一个我实现的例外支持做出设计思路、具体代码和结果分析。

2 系统调用例外

2.1 设计思路

根据实验指导书，首先需要明确，我们不需要一发生例外就立刻处理，这就理解“精确例外”这个概念：

简单来说就是，无论是算术溢出、地址错还是 syscall，都要先在译码级发现，再把异常信息随流水寄存器一路传到 WB，在写回级统一写 CP0 并刷新前级。

我修改了之前的实验已经实现的 SYSCALL 例外支持，新的设计完全沿用了这套思想。

1. ID 阶段通过 inst_SYSCALL 识别 op=0、funct=0x0C 的指令后，立即生成 exc_syscall_id，并在 exc_code_id 中填入 EXC_CODE_SYS。
2. 这组 exc_valid, exc_code, bad_addr 被编码进 ID_EXE_bus，经过 EXE/MEM 逐级透传，直到 WB 才真正触发异常。
3. 写回级里的 CP0 逻辑（Status/CAUSE/EPC）参照书中实现：一旦 exc_commit 为真，就把 EPC 写成当前指令的 PC，CAUSE.exc_code 写成 EXC_CODE_SYS，STATUS.EXL 拉高，同时通过 cancel 信号把 IF/ID/EXE/MEM 清空，并用 exc_bus 把 PC 改为异常入口地址 EXC_ENTER_ADDR。
4. 由于异常始终附着在那条 syscall 指令上，之前完成的指令不会被回滚，之后尚未提交的指令会被冲掉，精确例外得以保证。

2.2 核心代码与说明

decode.v 中的识别与异常打包

Listing 1: SYSCALL

```
1 assign inst_SYSCALL = (op == 6'b000000) & (funct == 6'b001100);
```

```

2      ...
3      wire exc_syscall_id = ID_valid & inst_SYSCALL;
4      assign exc_valid_id = exc_syscall_id | exc_ri_id | exc_break | exc_adel_if;
5      assign exc_code_id  = exc_adel_if    ? `EXC_CODE_ADEL :
6          exc_break      ? `EXC_CODE_BP    :
7          exc_syscall_id ? `EXC_CODE_SYS   :
8          exc_ri_id      ? `EXC_CODE_RI    : 5'd0;
9      assign bad_addr_id  = exc_adel_if_curr ? pc :
10         exc_adel_if_jbr ? jbr_target_raw :
11         exc_ri_id       ? pc : 32'd0;

```

1. inst_SYSCALL 纯组合判断 syscall 的编码;
2. exc_syscall_id 确保只有本条指令有效时才产生异常;
3. exc_valid_id/ exc_code_id 把 syscall 与其他异常统一编码打包, 下游模块不用关心类型, 只按异常码处理;
4. bad_addr_id 对 syscall 来说为 0, 保留地址无意义, 但结构保持一致, 保证总线格式统一。

ID_EXE_bus 拼接

Listing 2: SYSCALL

```

1      assign ID_EXE_bus = { ...,
2          exc_valid_id, exc_code_id, bad_addr_id,
3          rf_wen, rf_wdest,
4          inst_add_ov, inst_sub_ov,
5          pc};

```

1. 异常三元组与 PC 一起进 EXE; EXE/MEM 没有对 syscall 做额外逻辑, 仅传递。

wb.v 的 CP0/异常提交

Listing 3: SYSCALL

```

1      wire exc_req  = exc_valid_i & WB_valid;
2      assign exc_commit = irq_req | exc_req;
3      assign exc_code_wb = irq_req ? `EXC_CODE_INT : exc_code_i;
4
5      assign cancel    = (exc_commit | eret_commit) & WB_over;
6      assign rf_wen     = wen & WB_over & ~exc_commit;
7
8      wire exc_valid    = exc_commit | eret_commit;
9      wire [31:0] exc_pc = exc_commit ? `EXC_ENTER_ADDR : cp0r_epc;
10     assign exc_bus     = {exc_valid, exc_pc};

```

1. exc_valid_i/exc_code_i 就是 decode 的输出, syscall 到 WB 时 exc_req 为真;

- exc_commit 使 STATUS.EXL 置 1、CAUSE.exc_code 写入 EXC_CODE_SYS、EPC 写入 pc，同时禁止本周周期写回，防止 syscall 把 \$v0 等寄存器污染；
- cancel 和 exc_bus 分别做流水刷新与引导 IF 跳转。

2.3 测试程序与结果分析

Listing 4: inst_pipeline.coe

```

1      ...
2      ; SYSCALL
3      0000000C

```

我重写了一个 coe 文件用来测试本次实验实现的例外，其中有 0000000C 来测试 SYSCALL:

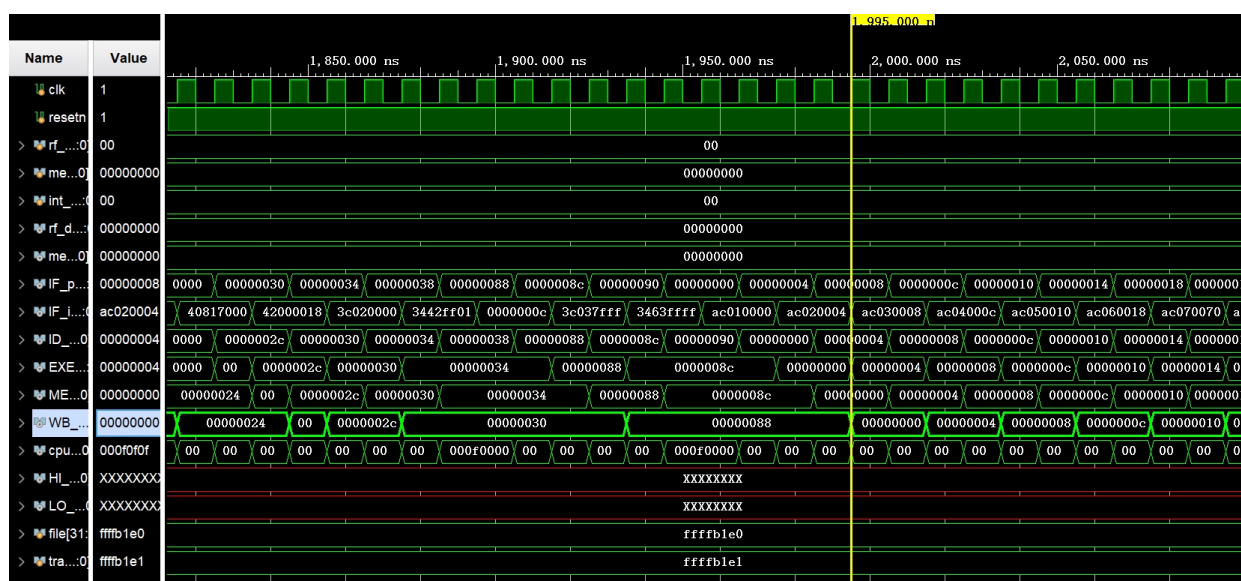


图 2.1: syscall 测试结果

可见在其他指令之后，顺序执行了 syscall，也顺利进入了 exception，最后也成功返回到了该指令的下一条指令继续执行，由于截图长度限制不再展示。

- IF_pc 走到 syscall 地址 (0x88) 后，ID_pc/EXE_pc/MEM_pc 依次对齐，说明流水在正常推进；
- 当 WB_pc = 0x88 时，下一拍立即跳为 00000000，这就是异常入口触发；
- 在异常服务程序里 (EX_JUMP_ADDR 区域) 执行若干指令后，再次回到 syscall 的下一条地址，流水恢复。

3 地址错例外

3.1 设计思路

根据实验指导书，我们把地址错分成两类：

1. 取指地址错 (IF ADEL), 需要在译码级就发现并阻止前端跳到非法地址;
2. 数据访存地址错 (LOAD/STORE ADEL/ADES), 在 MEM 阶段根据真实的 ALU 结果和字节使能判定。

对第一类, 我们在 decode 阶段同时检查“当前指令自身的 PC 是否未对齐”和“跳转/分支目标地址是否未对齐”; 一旦发现, 立即将异常附着在当前指令上, 且禁止 jbr_taken 驱动 PC, 这样非法目标不会进入流水。

对第二类, 我们在 MEM 阶段根据 ls_word 与 dm_addr[1:0] 判断是否未对齐, 并在异常标记后屏蔽 dm_wen, 防止写入无效数据。

这两种异常都随着流水寄存器一路传到 WB, 最终统一写 CP0 EPC/CAUSE/BadVAddr, 并通过 cancel/exc_bus 刷新前级, 保证精确异常。

3.2 核心代码与说明

decode.v 中的 IF ADEL 检查 .

Listing 5: IF ADEL

```

1  wire jbr_req      = (j_taken | br_taken);
2  wire [31:0] jbr_target_raw = j_taken ? j_target : br_target;
3  wire target_unaligned = jbr_req & |jbr_target_raw[1:0];
4  assign jbr_taken     = jbr_req & ID_over & ~target_unaligned;
5  assign jbr_target     = jbr_target_raw;
6
7  wire exc_adel_if_curr = ID_valid & |pc[1:0];
8  wire exc_adel_if_jbr  = ID_valid & jbr_req & target_unaligned;
9  assign exc_adel_if    = exc_adel_if_curr | exc_adel_if_jbr;
10 assign bad_addr_id     = exc_adel_if_curr ? pc :
11                        exc_adel_if_jbr ? jbr_target_raw :
12                        exc_ri_id       ? pc : 32'd0;

```

1. target_unaligned 只在存在跳转请求时才检测目标地址低两位;
2. jbr_taken 被 target_unaligned 屏蔽, 非法目标不会更新 PC;
3. exc_adel_if_curr 负责检测“IF/ID 读取的指令自身 PC 未对齐”, exc_adel_if_jbr 负责“跳转目标未对齐”;
4. bad_addr_id 在不同来源下分别记录当前 PC 或跳转目标, 供 CP0.BadVAddr 保存。

mem.v 中的数据访存 ADEL/ADES 判定 .

Listing 6: ADEL/ADES

```

1  wire misalign_word = ls_word & |dm_addr[1:0];
2  wire exc_adel      = MEM_valid & inst_load & ~exc_valid_i & misalign_word;
3  wire exc_ades      = MEM_valid & inst_store & ~exc_valid_i & misalign_word;
4  wire exc_valid_mem = exc_valid_i | exc_adel | exc_ades;
5  wire [4:0] exc_code_mem = exc_valid_i ? exc_code_i :

```

```

6         exc_adel      ? `EXC_CODE_ADEL :
7         exc_ades      ? `EXC_CODE_ADES : 5'd0;
8     wire [31:0] bad_addr_mem = exc_valid_i ? bad_addr_i :
9         (exc_adel | exc_ades) ? dm_addr : 32'd0;
10
11     always @(*) begin
12         if (MEM_valid && inst_store && ~exc_ades) begin
13             if (ls_word) dm_wen <= 4'b1111;
14             else case (dm_addr[1:0]) ... endcase
15         end else dm_wen <= 4'b0000;
16     end

```

1. misalign_word 只在处理字型 load/store 时生效
2. exc_adel/exc_ades 只在本阶段没有早先异常时才生成，保证优先级传递；
3. 一旦未对齐，dm_wen 被置 0、防止写 RAM，load_result 也不会写回；
4. 异常信息继续随 MEM_WB_bus 进入 WB，由 CP0 统一处理。

wb.v 的异常提交逻辑 同 syscall，当 exc_code_i == EXC_CODE_ADEL 或 EXC_CODE_ADES 时，写回级会写 EPC= 出错指令 PC，CAUSE.exc_code=4/5，BadVAddr= 对应地址，并通过 exc_bus 强制 IF PC 跳到异常入口。

3.3 测试程序与结果分析

Listing 7: inst_pipeline.coe

```

1     ...
2     ; IF ADEL
3     24050003    ; addiu r5,r0,3
4     00A00008    ; jr r5

```

这是 coe 文件用来测试 IF ADEL 的指令，00A00008 需要跳向的地址 0x3 就是未对齐地址，所以该指令就是预期触发 IF ADEL 例外处理的指令。仿真结果如下：



图 3.2: IF ADEL 测试结果

仿真中可以看到:当 jr r5 位于 ID 阶段时,target_unaligned 置 1,jbr_taken 被抑制,因此 IF_pc 并未跳到 0x3; 而该指令一路推进到 WB 时触发异常,IF_pc 跳到 0x00000000, BadVAddr=0x00000003。处理结束后 eret 返回 jr 的下一条,流水恢复,证明 IF ADEL 捕获成功。

Listing 8: inst_pipeline.coe

```

1      ...
2      ; ADEL
3      24080005      ; r8 = 5
4      8D090001      ; lw r9,1(r8)  -> Load ADEL
5      ; ADES
6      AD090002      ; sw r9,2(r8)  -> Store ADES

```

这是 coe 文件用来测试 ADEL 和 ADES 的指令, 仿真结果分别如下:

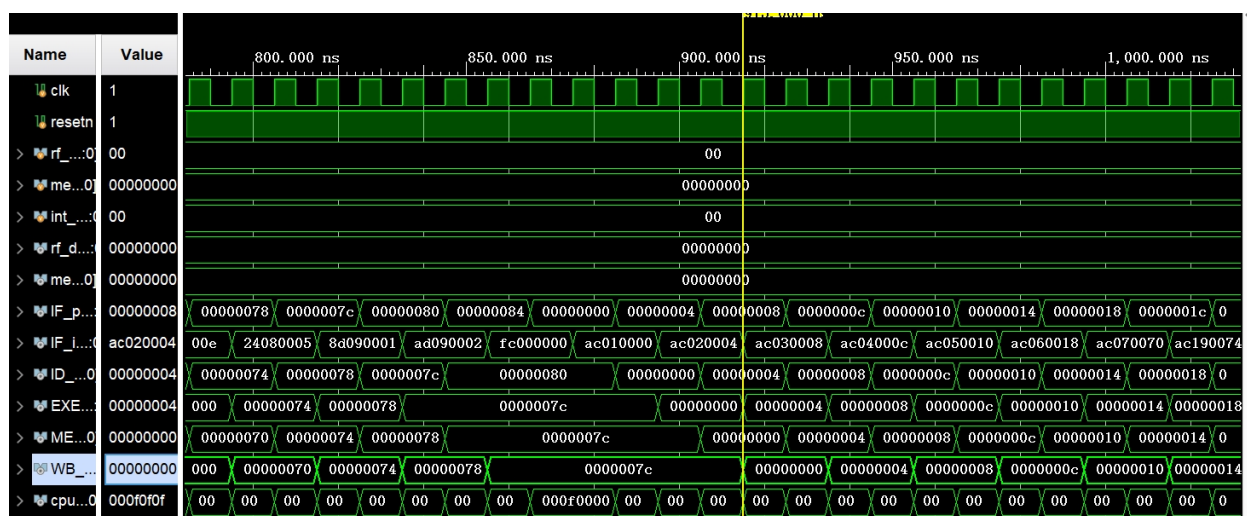


图 3.3: ADEL 测试结果


```

12         rf_wen, rf_wdest,
13         inst_add_ov, inst_sub_ov,
14         pc};

```

1. 在 opcode/funct 级别区分 ADD/SUB 与 ADDU/SUBU;
2. inst_add/inst_sub 控制 ALU 运算本体, 使 ADD/SUB 实际执行加/减;
3. inst_add_ov/inst_sub_ov 两位标志被编码进 ID_EXE_bus, 供执行阶段判断溢出。

exe.v 执行阶段溢出检测

Listing 10: OV

```

1  wire [31:0] ov_b    = inst_sub_ov ? (~alu_operand2 + 1'b1) : alu_operand2;
2  wire [31:0] ov_sum  = alu_operand1 + ov_b;
3  wire ov_signed = (inst_add_ov | inst_sub_ov) &
4                  ((~alu_operand1[31] & ~ov_b[31] & ov_sum[31]) |
5                   (alu_operand1[31] & ov_b[31] & ~ov_sum[31]));
6
7  wire exc_overflow = EXE_valid & ~exc_valid_i & ov_signed;
8  wire exc_valid_exe = exc_valid_i | exc_overflow;
9  wire [4:0] exc_code_exe = exc_valid_i ? exc_code_i :
10                      exc_overflow ? `EXC_CODE_OV : 5'd0;
11 wire [31:0] bad_addr_exe = exc_valid_i ? bad_addr_i : 32'd0;
12
13 assign EXE_MEM_bus = {...,
14                      exc_valid_exe, exc_code_exe, bad_addr_exe,
15                      rf_wen, rf_wdest,
16                      pc};

```

1. ov_b/ov_sum 模拟有符号加减: 对 SUB 先取补再加;
2. ov_signed 对符号位进行检测: 正数 + 正数得负, 或负数 + 负数得正即为溢出;
3. exc_overflow 只在当前流水没有早期异常、且本条指令有效时置 1;
4. 一旦溢出, 异常三元组被替换为 EXC_CODE_OV, 并随 EXE→MEM 总线继续传递。

其他 mem.v/wb.v 无需额外处理, 只需透传异常; 在 WB 阶段 exc_commit 会写 EPC= 该 ADD/SUB 的 PC, CAUSE.exc_code=0x0C, 流水被刷新, 写回被屏蔽。

4.3 测试程序与结果分析

Listing 11: inst_pipeline.coe

```

1  ...
2  3C037FFF      ; lui   r3,0x7FFF
3  3463FFFF      ; ori   r3,r3,0xFFFF -> r3 = 0x7FFFFFFF
4  00632020      ; add   r4,r3,r3      -> 正溢出

```

仿真时观察到:

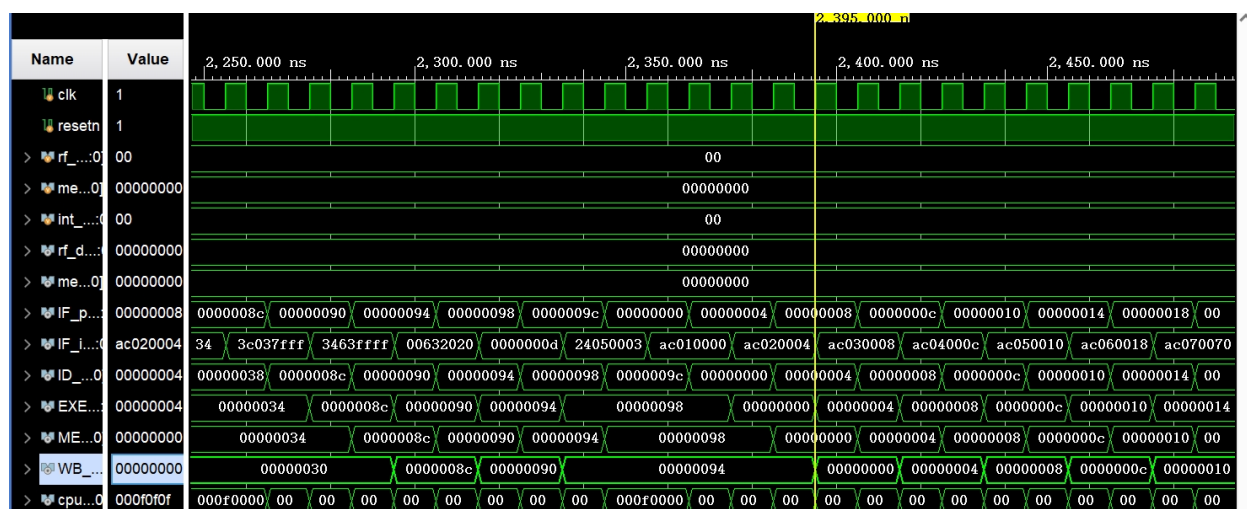


图 4.5: ov 测试结果

可见当 add r4,r3,r3 进入 EXE 阶段时, ov_signed 置 1; 推进到 WB 时, exc_bus 在该拍拉高, PC 跳到异常入口 0x00000000; 异常处理程序结束后 eret 返回到加法的下一条指令, 后续流水继续运行。

5 断点例外

5.1 设计思路

根据实验指导书, break 被定义为一种“程序员显式插入的调试例外”, 实现方式与 syscall 类似: 在译码级识别 break, 把异常信息随流水传递, 在写回级统一提交并跳转到异常入口。

我的实现沿用了这套精确例外框架。

译码阶段根据 funct=0x0D 判定 BREAK, 设置 exc_break, 并在异常编码 exc_code_id 中填入 EXC_CODE_BP。

之后该异常标记随 ID_EXE_bus→EXE_MEM_bus→MEM_WB_bus 一路透传;

写回级收到 EXC_CODE_BP 时写 EPC/CAUSE、置 STATUS.EXL 并刷新流水, 从而在断点指令提交时准确跳转到异常入口, 调试器可在该入口读取 EPC/CAUSE 决定后续动作。

5.2 核心代码与说明

译码阶段识别与打包

Listing 12: BP

```

1  assign inst_BREAK = op_zero & (funct == 6'b001101);
2  ...
3  assign exc_break   = ID_valid & inst_BREAK;
4  assign exc_valid_id = exc_syscall_id | exc_ri_id | exc_break | exc_adel_if;
5  assign exc_code_id = exc_adel_if ? `EXC_CODE_ADEL :
6      exc_break ? `EXC_CODE_BP :
7      exc_syscall_id? `EXC_CODE_SYS :

```

```
8      exc_ri_id      ? `EXC_CODE_RI      : 5'd0;
```

1. inst_BREAK 只在 opcode=0、funct=0x0D 时为真；
2. exc_break 附着在当前 ID 指令上，只有在 ID_valid 且确实是 BREAK 时才触发；
3. exc_code_id 将断点编码为 EXC_CODE_BP，与其他异常统一格式打包。

流水寄存器传递

Listing 13: BP

```
1      assign ID_EXE_bus = {...,
2          exc_valid_id, exc_code_id, bad_addr_id,
3          rf_wen, rf_wdest,
4          inst_add_ov, inst_sub_ov,
5          pc};
```

exc_valid_id/exc_code_id 一路随总线进入 EXE/MEM/WB；断点不需要额外的 EXE/MEM 逻辑。

写回级统一提交

Listing 14: BP

```
1      wire exc_req  = exc_valid_i & WB_valid;
2      assign exc_commit  = irq_req | exc_req;
3      assign exc_code_wb = irq_req ? `EXC_CODE_INT : exc_code_i;
4
5      assign cancel     = (exc_commit | eret_commit) & WB_over;
6      assign rf_wen     = wen & WB_over & ~exc_commit;
7
8      wire [31:0] exc_pc = exc_commit ? `EXC_ENTER_ADDR : cp0r_epc;
9      assign exc_bus     = {exc_commit | eret_commit, exc_pc};
```

1. 当 exc_code_i==EXC_CODE_BP 且 WB_valid，exc_req 为真，exc_commit 在该指令提交时触发；
2. EPC 记录断点指令的 PC，CAUSE.exc_code 写成 EXC_CODE_BP，STATUS.EXL 置 1；
3. cancel 清空前级并阻止写回，exc_bus 告诉 IF 跳到异常入口 0x0。断点处理完 eret 返回，继续执行下一条。

5.3 测试程序与结果分析

Listing 15: inst_pipeline.coe

```
1      ...
2      ; BREAK
3      0000000D
```

仿真时观察到:

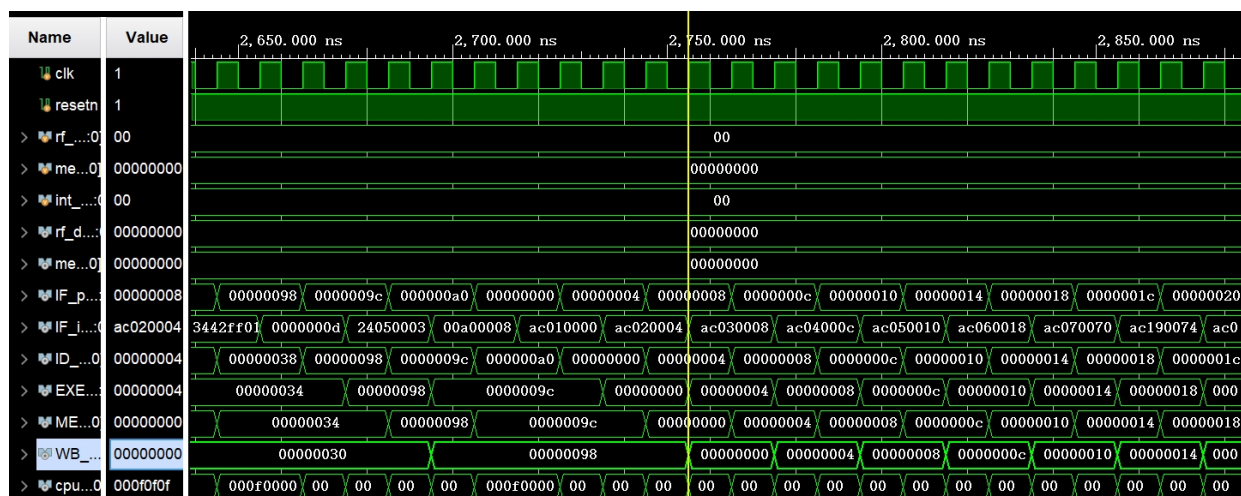


图 5.6: break 测试结果

可见:

1. 当 IF_pc 走到断点地址 (0x98) 时, ID_pc/EXE_pc/MEM_pc 依次对齐, WB_pc 在该指令提交前一拍与其相等;
2. 紧接着下一拍, IF_pc 跳到 0x00000000 (异常入口)
3. CP0.CAUSE 在该周期写入 ExcCode, EPC= 断点 PC; 异常服务程序可根据 CAUSE/EPC 执行调试操作, 最终 eret 返回断点下一条。
4. 由于断点指令没有写寄存器/访存, 写回阶段禁止写回后寄存器状态保持不变, 流水按期望在断点停下、进入异常入口, 再由异常程序决定恢复或单步。

6 保留指令例外

6.1 设计思路

根据实验指导书, 译码阶段如果遇到未实现或非法的指令编码, 应立即标记“保留指令异常”(Reserved Instruction), 并按照精确异常流程在写回阶段统一提交, 防止非法指令对状态造成影响。

我的实现遵循这一策略:

在 decode 中显式列出所有受支持的指令, 将它们按 OR 组合成 inst_supported;

只要当前指令有效且不在支持列表内, 就置 exc_ri_id, 同时在 exc_code_id 中填入 EXC_CODE_RI。

异常信息随流水寄存器一路传递, 不做其他处理;

写回级检测到该异常后写 EPC/CAUSE、刷流水、跳异常入口。

这样即使遇到随机的非法指令, 也能保持精确异常, 异常处理程序可以查看 EPC/CAUSE 决定如何处理, 不过本次实验我没有修改异常处理程序。

6.2 核心代码与说明

decode.v 中的支持列表与异常标记

Listing 16: RI

```

1  assign inst_supported = inst_ADDU | inst_SUBU | ... | inst_BREAK;
2  assign exc_ri_id      = ID_valid & ~inst_supported;
3  assign exc_valid_id   = exc_syscall_id | exc_ri_id | exc_break | exc_adel_if;
4  assign exc_code_id    = exc_adel_if ? `EXC_CODE_ADEL :
5      exc_break        ? `EXC_CODE_BP      :
6      exc_syscall_id ? `EXC_CODE_SYS      :
7      exc_ri_id        ? `EXC_CODE_RI      : 5'd0;
8  assign bad_addr_id    = exc_adel_if_curr ? pc :
9      exc_adel_if_jbr  ? jbr_target_raw :
10     exc_ri_id         ? pc : 32'd0;

```

1. inst_supported 将所有支持的 R/I/J 型指令与 CP0 操作列出，未列出的编码自动视为 RI;
2. exc_ri_id 只在当前流水指令有效且不在支持列表时置 1;
3. exc_code_id 根据优先级挑选异常码，RI 排在 syscall/地址错之后;
4. bad_addr_id 对 RI 填当前 PC，便于异常处理程序查看出错地址。

流水寄存器与写回级处理

流水寄存器与写回级处理与其他异常共用：

ID_EXE_bus 将异常三元组打包, EXE/MEM 不做更改, WB 阶段统一提交, 写 EPC/CAUSE=10 (RI), 清空流水, 跳异常入口。

此处不再赘述。

6.3 测试程序与结果分析

Listing 17: inst_pipeline.coe

```

1  ...
2  ; RI
3  FC000000

```

仿真时观察到:

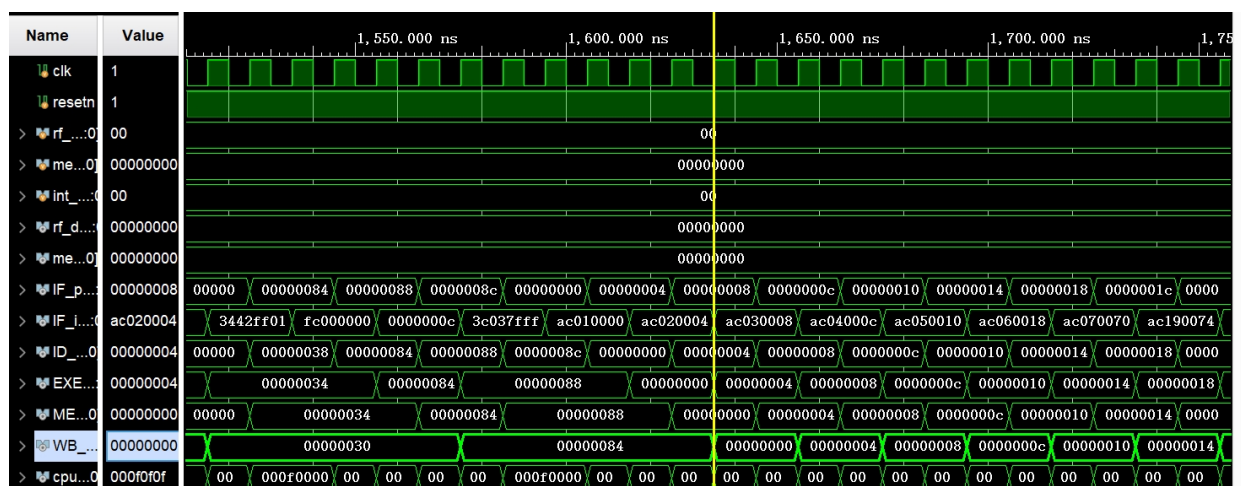


图 6.7: RI 测试结果

这条指令不属于支持列表（既不是合法 R/I/J，也不是 CP0 操作），译码阶段立刻将 exc_ri_id 置 1。仿真波形表明：

1. 当 IF_pc 执行到 FC000000 时，ID_pc/EXE_pc/MEM_pc 依次对齐，说明指令沿流水推进；
2. 到达写回阶段的下一拍，IF_pc 跳到异常入口 0x00000000；
3. CP0.CAUSE.exc_code 记录为 0x0A（RI），EPC= 该非法指令地址。异常程序在入口处即可读取 EPC/CAUSE 进行处理；
4. 异常处理结束后 eret 返回到 RI 的下一条指令，后续执行不受影响。

7 中断例外

7.1 设计思路

实验要求在 CP0 中实现可屏蔽的外部/定时器中断，并把中断例外纳入五级流水的精确异常框架。我在 WB 级实现了 STATUS/CAUSE/EPC/Count/Compare 寄存器以及中断仲裁逻辑。

在每个周期，WB 都会根据 int_in(外部中断输入)和 Count/Compare 的计时器状态更新 Cause.IP，再结合 Status.IM/IE/EXL 判断是否要把当前写回指令标记为“中断例外”。

一旦满足条件 (IE=1、EXL=0、某一 IP 位与 IM 位同时置位)，irq_req 置 1，写回级就会写 EPC= 当前 PC、CAUSE.exc_code=0 (EXC_CODE_INT)、STATUS.EXL=1，并通过 cancel/exc_bus 刷新流水、跳转到异常入口。

由于中断是在写回阶段统一提交，之前的指令都已经退出，之后的指令全部被冲掉，仍然满足精确异常要求。

Count/Compare 模块则实现了“定时器中断”：Count 每拍自增，Compare 可通过 mtc0 编程；当 Count==Compare 时 Cause.IP7 置 1，写 Compare 会清除 TI 位。

7.2 核心代码与说明

CP0 状态寄存器与计时器

Listing 18: wb.v

```

1  reg status_exl_r, status_ie_r;
2  reg [7:0] status_im_r;
3  reg [7:0] cause_ip_r;
4  reg [31:0] count_r, compare_r;
5
6  wire status_wen = mtc0 & (cp0r_addr=={5'd12,3'd0});
7  ...
8  always @(posedge clk) begin
9      if (!resetn || eret_commit) begin
10         status_exl_r <= 1'b0;
11         status_ie_r  <= 1'b0;
12         status_im_r  <= 8'd0;
13     end else if (exc_commit) begin
14         status_exl_r <= 1'b1;
15     end else if (status_wen) begin
16         status_ie_r  <= mem_result[0];
17         status_exl_r <= mem_result[1];
18         status_im_r  <= mem_result[15:8];
19     end
20 end
21
22 always @(posedge clk) begin
23     if (!resetn) count_r <= 32'd0;
24     else if (count_wen) count_r <= mem_result;
25     else count_r <= count_r + 1'b1;
26 end
27
28 always @(posedge clk) begin
29     if (!resetn) compare_r <= 32'd0;
30     else if (compare_wen) compare_r <= mem_result;
31 end

```

1. CP0 寄存器写在 wb.v 中，STATUS 仅实现 EXL/IE/IM 三组字段，且只能由异常/mtc0 更新；
2. Count/Compare 形成一个简单的定时器，Count 自动递增，Compare 通过软件写入目标值。

Cause.IP 更新与中断仲裁

Listing 19: wb.v

```

1  always @(posedge clk) begin
2      if (!resetn) begin
3          cause_ip_r <= 8'd0;
4      end else begin
5          cause_ip_r[5:0] <= int_in; // 外部中断
6          if (compare_r!=32'd0 && count_r==compare_r)
7              cause_ip_r[7] <= 1'b1; // TI 位
8          else if (compare_wen)
9              cause_ip_r[7] <= 1'b0;

```



```

10     end
11 end
12
13 assign eret_commit = eret & WB_valid;
14 wire [7:0] int_pending = cause_ip_r & status_im_r;
15 wire irq_req  = status_ie_r & ~status_exl_r & |int_pending & WB_valid;
16 wire exc_req  = exc_valid_i & WB_valid;
17 assign exc_commit  = irq_req | exc_req;
18 assign exc_code_wb = irq_req ? `EXC_CODE_INT : exc_code_i;

```

1. Cause.IP 前 6 位反映外部中断输入 int_in，第 7 位为定时器 TI;
2. 写 Compare 自动清除 TI;
3. irq_req 判断 IM/IE/EXL，并与 |int_pending 即某一 IM/IP 同时为 1 的情况结合，只要有中断请求且写回指令有效，就抢占异常源;
4. exc_code_wb 因此在中断时为 EXC_CODE_INT (0)，其余异常沿用原码。

异常提交与流水刷新

Listing 20: wb.v

```

1 assign cancel  = (exc_commit | eret_commit) & WB_over;
2 assign rf_wen  = wen & WB_over & ~exc_commit;
3
4 wire          exc_valid = exc_commit | eret_commit;
5 wire [31:0] exc_pc     = exc_commit ? `EXC_ENTER_ADDR : cp0r_epc;
6 assign exc_bus  = {exc_valid, exc_pc};

```

与其它异常共用同一套写 EPC/CAUSE、刷新流水的机制,中断返回靠 eret,Status.EXL 在 eret_commit 时清零。

7.3 测试程序与结果分析

Listing 21: inst_pipeline.coe

```

1 ...
2 ; Count/Compare
3 3C020000
4 3442FF01
5 40826000      ; mtc0 r2,$12  (Status: IM 全开、IE=1)
6 24020000
7 40824800      ; mtc0 r2,$9   (Count=0)
8 2402000A
9 40825800      ; mtc0 r2,$11  (Compare=10)
10 ; normal inst
11 24010001
12 00011100

```

```

13      00411821
14      00022082
15      28990005
16      00642823
17      AC050014
18      00A23027
19      00C33825
20      00E64026

```

我在 40825800 后设置了多条一般指令，因为 Compare 设置为 10，那么 10 个时钟周期后预期会触发中断例外，又因为我们的 cpu 设置一周期为 10ns，那么预期就是 100ns 后正在发生的指令会触发时钟中断。进行仿真，观察到：

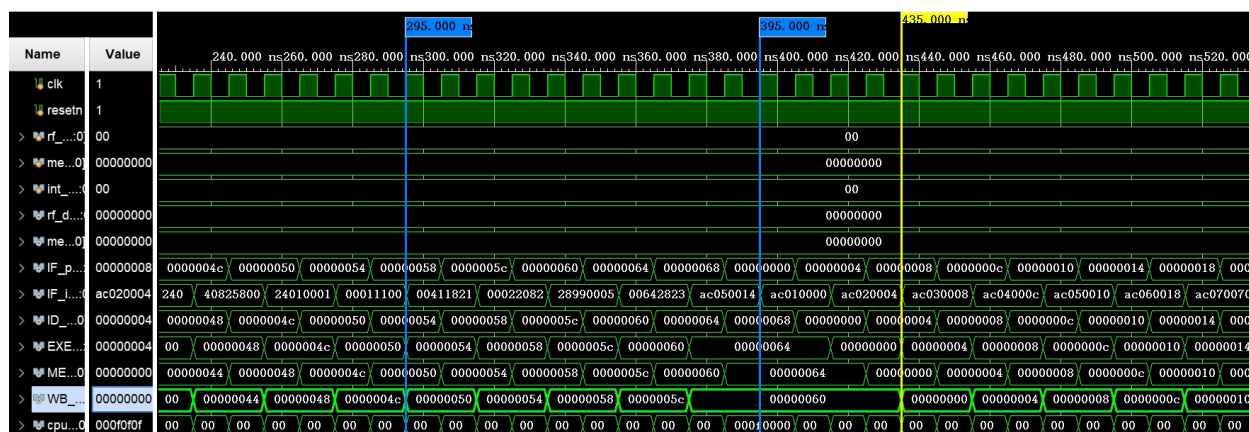


图 7.8: 时钟中断测试结果

我们可以在波形图中找到 40825800 对应位置，对应 pc 为 0x4c，该指令 wb 执行完后处在 295ns，向后数 10 个周期也就是 100ns 来到 395ns，此时 wb 阶段指令为 28990005 是个普通的 slti 指令，立刻发生了中断进入了 exception，说明了中断例外按照预期成功实现。

8 总结感想

这次补齐例外实验，让我对五级流水里“精确异常”的概念有了更直观的理解。系统调用、断点、地址错、保留指令、整形溢出、中断这些异常虽然触发条件各不相同，但它们都依赖同一套流程：在译码/访存阶段尽早标记，把异常三元组沿着流水寄存器带到写回级，最后由 CP0 统一写 EPC/CAUSE、刷新前级。只要这一套路子梳理清楚，其实每种异常的实现都不过是在相应阶段插入一条“异常标志”的组合逻辑而已。

另一个收获是对 CP0 寄存器的掌握。本实验把 Status/CAUSE/EPC/BadVAddr 以及 Count/Compare 都落地实现，中断尤其是 Count/Compare 的定时器中断，需要软件和硬件配合：软件负责配置 IM/IE 和 Compare 值，硬件在 WB 级检测 IP/IM，并在合适时机触发 EXC_CODE_INT。中断不是单纯靠 int_in 拉高就行，还要考虑 STATUS_EXL 的状态、异常返回的过程。这种硬件/软件配合，在做测试指令的设计、调整异常顺序时也能体会到。